

AFFORDABLE FOOTBALL ANALYTICS: A CONSUMER-HARDWARE
BENCHMARK OF LIGHTWEIGHT YOLOV8 DETECTION MODELS

A THESIS SUBMITTED TO
THE FACULTY OF ARCHITECTURE AND ENGINEERING
OF
EPOKA UNIVERSITY

BY

OLTIAN ELEZI

IN PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR BACHELOR DEGREE
IN
COMPUTER ENGINEERING

JUNE 2026

I hereby declare that all information in this document has been obtained and presented in accordance with academic rules and ethical conduct. I also declare that, as required by these rules and conduct, I have fully cited and referenced all material and results that are not original to this work.

Name Surname: Oltian Elezi

Signature: _____

ABSTRACT

AFFORDABLE FOOTBALL ANALYTICS: A CONSUMER-HARDWARE BENCHMARK OF LIGHTWEIGHT YOLOV8 DETECTION MODELS

Elezi, Oltian

B.Sc., Department of Computer Engineering

Supervisor: Dr. Edlir Spaho

Football today is being evolving rapidly. This evolution is largely driven by implementation of performance analysis through technology, which the biggest clubs use to great effect. However, these methods are far too expensive for the vast majority of clubs and academies in the world, and are therefore inaccessible to them. This thesis addresses that issue by evaluating the current state-of-the-art technologies in object detection inside the football domain, and offering the best option for different practitioners spanning different budgets of consumer-grade hardware configurations.

The three lightest variants of the YOLOv8 architecture are fine-tuned pre-trained weights on the Bundesliga Data Shootout dataset, a publicly available dataset which consists of annotated frames from broadcast footage from football games with annotations covering four different classes: player, ball, goalkeeper, referee.

All three models are trained on identical conditions, and then evaluated on their accuracy and inference speed using standard metrics such as mAP@0.5 for accuracy and frames per second for inference speed. After a thorough analysis of the experiment and results is made, the findings are used to offer practical guidance for different practitioners and their goals, based on their hardware tier.

Keywords: *Computer Vision, Football Analytics, Multi-Object Tracking, Real-Time Video Processing, Object Detection*

ACKNOWLEDGMENTS

I would like to express my sincere gratitude to my supervisor, Dr. Edlir Spaho, for their invaluable guidance, support, and encouragement throughout my research.

I am also grateful to my family and friends for their continuous support and understanding during my studies.

Finally, I would like to thank Epoka University for providing me with the opportunity to pursue my graduate studies and for the excellent facilities and resources.

LIST OF ABBREVIATIONS

YOLO	You Only Look Once
HOG	Histogram of Oriented Gradients
SVM	Support Vector Machine
CNN	Convolutional Neural Network
R-CNN	Region-based Convolutional Neural Network
GPU	Graphics Processing Unit
CPU	Central Processing Unit
mAP	Mean Average Precision
IoU	Intersection over Union
FPS	Frames per Second
CIoU	Complete Intersection over Union
TPU	Tensor Processing Unit
FLOP	Floating Point Operation
SOC	System-on-Chip
MPS	Metal Performance Shaders
CUDA	Compute Unified Device Architecture

TABLE OF CONTENTS

ABSTRACT	ii
ACKNOWLEDGMENTS	iii
LIST OF ABBREVIATIONS	iv
LIST OF TABLES	viii
LIST OF FIGURES	ix
CHAPTER 1	1
INTRODUCTION	1
1.1 Introduction	1
1.2 Background and Motivation	1
1.3 Problem Statement	2
1.4 Research Objectives	3
1.5 Contributions	4
1.6 Thesis Organization	4
CHAPTER 2	6
LITERATURE REVIEW	6
2.1 Introduction	6
2.2 Theoretical Background	6
2.2.1 From Traditional Methods to Deep Learning Detection	6
2.2.2 Single-Stage detection: The YOLO paradigm	7
2.2.3 Target Performance Metrics	8
2.3 Related Work	9
2.3.1 The Evolution of the YOLO Architecture	9
2.3.2 YOLOv8 Variant Comparisons	9
2.3.3 YOLO Based Detection in Football Video Footage	11
2.3.4 YOLOv8 Variant Scaling	11
2.3.5 The Bundesliga Data Shootout dataset	12
2.4 Positioning of the Present Study - Summary	13

CHAPTER 3	14
METHODOLOGY	14
3.1 Introduction.....	14
3.2 Research Design.....	14
3.3 Dataset	15
3.3.1 Source	15
3.3.2 Dataset Splitting	15
3.3.3 Data Augmentation	15
3.4 Training Procedure	17
3.5 Hardware Configurations	17
3.6 Inference Benchmarking.....	19
3.6.1 Settings.....	19
3.6.2 Latency Measurement	19
3.7 Summary	19
CHAPTER 4	21
RESULTS AND DISCUSSION	21
4.1 Training Results	21
4.1.1 Training Convergence	21
4.1.2 Detection Metrics.....	22
4.1.3 Class Detection Performance	23
4.1.4 Precision-Recall Curves	23
4.1.5 Confusion Matrices.....	25
4.1.6 Training Summary	27
4.2 Inference Benchmarks	27
4.2.1 Device A – Discrete NVIDIA GeForce RTX 3060 12GB GPU (CUDA)	28
4.2.2 Detection Volume and Confidence.....	28
4.2.3 Device B – Apple MacBook Pro M4 (MPS)	30
4.2.4 Device C – Lenovo ThinkPad T490s (CPU only).....	32
4.3 Summary	34
CHAPTER 5	36

CONCLUSION AND FUTURE WORK	36
5.1 Conclusion.....	36
5.2 Limitations	37
5.3 Limitations	37
5.4 Future Work	38
5.5 Final Remarks	38
REFERENCES	39

LIST OF TABLES

2.1	YOLOv8 model variants and their performance metrics (Ultralytics, 2023) [18].	12
3.1	Augmentation configuration applied to the dataset from default Roboflow configurations.	17
3.2	Shared training configuration applied identically to all three YOLOv8 variants.	18
3.3	Hardware specifications of the three consumer-grade devices used for inference benchmarking.	19
4.1	Best-epoch validation metrics for all three model variants.	22
4.2	Per-class average precision (AP) for all three model variants.	23
4.3	Inference summary statistics for all three YOLOv8 variants on Device A (RTX 3060, CUDA).	28
4.4	Per-class detection counts and proportions for all three YOLOv8 variants on Device A (RTX 3060, CUDA).	29
4.5	Confidence score distribution across bins for all three YOLOv8 variants on Device A (RTX 3060, CUDA).	29
4.6	Inference summary statistics for all three YOLOv8 variants on Device B (Apple M4, MPS).	30
4.7	Per-class detection counts and proportions for all three YOLOv8 variants on Device B (Apple M4, MPS).	31
4.8	Confidence score distribution across bins for all three YOLOv8 variants on Device B (Apple M4, MPS).	32
4.9	Inference summary statistics for all three YOLOv8 variants on Device C (Lenovo ThinkPad T490s, CPU only).	33
4.10	Per-class detection counts and proportions for all three YOLOv8 variants on Device C (Lenovo ThinkPad T490s, CPU only).	33
4.11	Confidence score distribution across bins for all three YOLOv8 variants on Device C (Lenovo ThinkPad T490s, CPU only).	34

LIST OF FIGURES

1.1	Wearable sensors which allow detailed player position tracking.....	3
2.1	System overview of early R-CNN object detection (Girshick et al., 2014 [6].....	7
2.2	Architecture of the first YOLO model (Redmon et al., 2016) [12]	7
2.3	A comparison between YOLOv8 and other YOLO models (Ultralytics, 2023) [18]	10
3.1	An example of an annotated and augmented image in our dataset [5] ...	16
4.1	Training curves for all three models during 50 epochs	22
4.2	Precision-Recall curve for the YOLOv8n model	24
4.3	Precision-Recall curve for the YOLOv8s model	24
4.4	Precision-Recall curve for the YOLOv8m model	25
4.5	Confusion matrix for the YOLOv8n model.....	26
4.6	Confusion matrix for the YOLOv8s model	26
4.7	Confusion matrix for the YOLOv8m model	27
4.8	Confidence distribution for all models on Device A(RTX 3060, CUDA). 30	
4.9	Confidence distribution for all models on Device B(MacBook Pro M4, MPS).	32
4.10	Confidence distribution for all models on Device C(Lenovo Thinkpad T490s, CPU).	34

CHAPTER 1

INTRODUCTION

1.1 Introduction

Football is widely renowned as the world's game, yet throughout history the processes used to identify and promote talent have largely relied on subjective human judgement and limited networks of knowledge. In times where data driven solutions are being implemented in all fields to improve efficiency and performance, football is no different, and as such it has been fostering a revolution in data analysis and statistical decision making.

This thesis aims to bring the two sides together, that is, to find efficient and cost-effective ways to bring this statistical revolution in football to all the footballers in the world. By exploring already developed technologies and pipelines and implementing them in very accessible environments we can evaluate the possibility of advancing the whole world of football players and academies, not just the richest of the rich.

Concretely, we will not only study the already well-researched implementation of YOLOv8 models in generating football statistics from football broadcast footage, but also look to find a balance between the accuracy and performance of each model in different levels of consumer-grade computational hardware.

1.2 Background and Motivation

During recent times, the football industry has experienced massive growth driven by the increasing broadcasting revenues, sponsorship agreements and competition prizes. As a result, the clubs are more incentivized than ever to form competitive squads and optimize their performance as much as possible in order to remain competitive in a very cut-throat field. In times where the advancements in technology are making it influential for every sector, football is no different, and the market for data driven tools that can supplement every aspect of improving performance on the field is growing in parallel with the sport.

A key part of improving performance on the pitch is finding the best players.

However, due to the global nature of the sport, spanning all continents, attracting passionate players of all cultural, ethnical and economic backgrounds, being picked as a talent often resorts more to opportunity and exposure rather than pure talent and ability. As such, players from wealthier backgrounds or more advanced countries benefit from that exposure, which poses a disadvantage for the others and makes a gap in the player networks for the clubs and academies.

Another important aspect in forming great football teams is the need to objectively evaluate players. Through statistics, we can measurably evaluate the production and contribution of players on the field and therefore remove all kinds of biases that the human eye of a scout can be prone to.

1.3 Problem Statement

Implementing statistics and data analysis in football is far from a novel concept, in fact, it has already been implemented at the top level with huge success. In the early days, football statistics were generated through meticulous manual labour in data entry (Memmert and Raabe, 2018) [11], before progressing into data driven methodologies based on positional information and computational analysis. This progress has enabled clubs to achieve the objective and measurable structures of information that allow them to optimize their player acquisition.

Brighton & Hove Albion F.C., playing in the top division of English Football are one of the clubs leading this statistical revolution in modern football. The club’s Technical Director, David Weir, attributed their rise to their extensive statistical information across many leagues worldwide (Kershaw, 2025) [8].

The collection of data is the most important and difficult aspect of this revolution. The wealthiest clubs such as the aforementioned Brighton F.C. make use of very advanced, and therefore, expensive pipelines (such as wearable sensors worn by players) to truly achieve the level of statistical accuracy a club at the top level truly needs in order to reliably produce results in an extremely very cut-throat environment. However, being “the world’s game”, football spans players and clubs of all sorts of economic backgrounds, most of which are not capable of implementing the same pipelines. This is where computer vision systems offer an accessible option, as they require only video footage coupled with a capable computing device.

That capability varies across consumers and pipelines, and constitutes the domain of our study. This thesis investigates the accuracy-speed trade-off of the three lightest YOLOv8 models by evaluating their performance on three consumer level

machines each with different capabilities. This should help further users find the best performing model for their environment and specific goals.

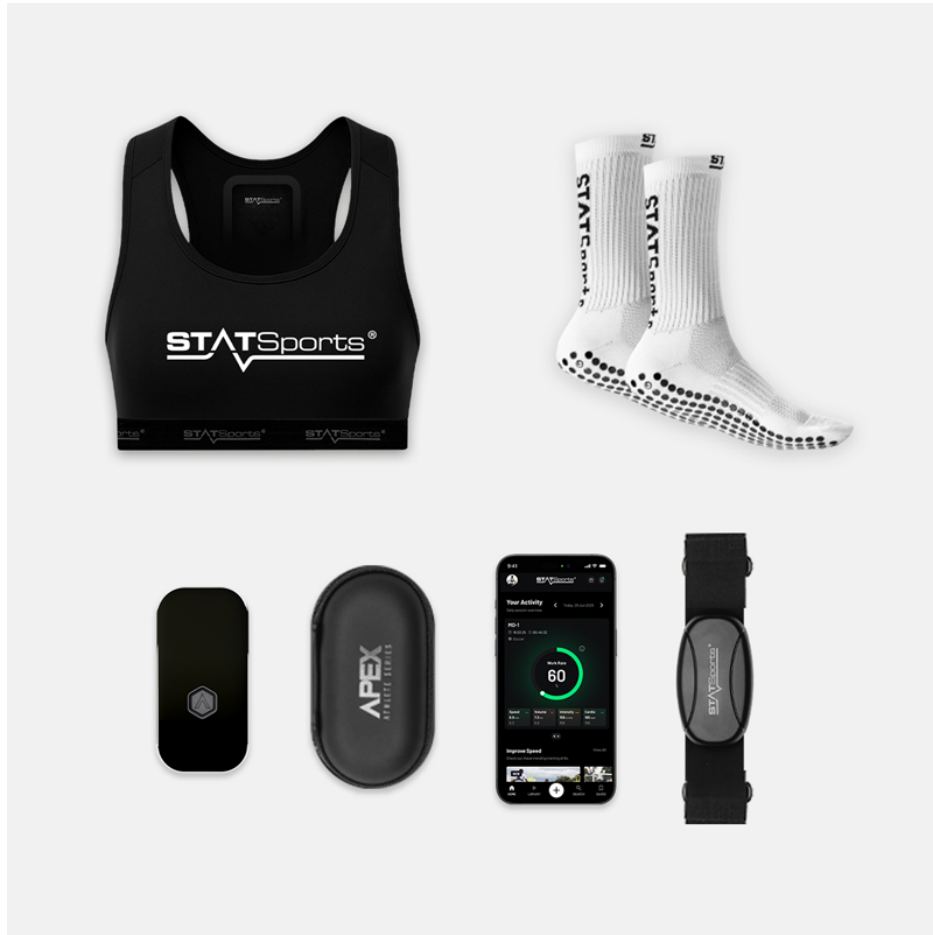


Figure 1.1. Wearable sensors which allow detailed player position tracking.

1.4 Research Objectives

The main goal of this thesis is to evaluate the three lightest YOLOv8 models across three different levels of hardware configurations, all accessible by average consumer such as individuals, small clubs or football academies. More specifically this separates into:

- Fine-tuning the YOLOv8n, YOLOv8s, and YOLOv8m models on the Bundesliga Data Shootout dataset under measurable and identical training conditions
- Evaluating the accuracy of each model using standard metrics in the field such as mAP@0.5, precision and recall.

- Benchmark the inference speed performance of each model across each of our selected hardware configurations by using the frames per second metric.
- Analyze the accuracy-speed trade-off of each model in each device, and recommend a model for each hardware level.

1.5 Contributions

The primary contributions of this thesis are summarized as follows:

1. A systematic empirical comparison of YOLOv8n, YOLOv8s, and YOLOv8m trained on the Bundesliga Data Shootout dataset, providing class-level detection performance benchmarks.
2. A hardware inference benchmark measuring the latency and throughput of each model variant across three consumer-grade hardware configurations at different price points, producing a dataset of performance results that reflects real-world deployment conditions rather than data center benchmarks.
3. A quantitative analysis of the trade-off between detection accuracy and inference speed across the three YOLOv8 variants, identifying at which point scaling up to a larger model no longer justifies the additional computational cost on consumer-grade hardware

1.6 Thesis Organization

This thesis is organized into five chapters, each focusing in a separate step of the research:

Chapter 2 evaluates the current progress in the field by studying the history of object detection going to the present state-of-the-art models and the YOLO architecture family. It also provides information on prior work in football video analytics and hardware constrained video inference benchmarks.

Chapter 3 explains in detail methodology. It describes the Bundesliga Data Shootout dataset which is used, the way it is pre-augmented, the controlled training procedure applied to each model identically and the three hardware configurations used for inference benchmarking.

Chapter 4 presents the results of our experiment and offers the comparison of each model across different accuracy metrics and inference speed measurements on each of the three hardware configurations.

Chapter 5 concludes the thesis through summarizing the key findings, recommending a model for each tier of hardware, discussing the limitations of our research, and pointing out directions for future research in affordable football video analytics.

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction

The growing demand for affordable analytics tools across all levels of football has attracted significant interest in research over the last few years. This effort is spearheaded by the YOLO (You Only Look Once) architecture for object detection, which has developed into a dominant architecture for real-time video object detection. This chapter reviews the current literature that provides the foundations for this thesis.

The review is organized as follows. Section 2.2 establishes the theoretical background by following the development of object detection from handcrafted feature description to the modern YOLO architecture. Section 2.3 surveys the related work, detailing the evolution of the YOLO architecture, its current implementation in football video object detection, prior studies on the benchmarks of models in hardware-constrained environments, and an overview of the dataset. Section 2.4 identifies the positioning of this study within the current literature, and Section 2.5 presents a summary of the major themes established in this review.

2.2 Theoretical Background

2.2.1 From Traditional Methods to Deep Learning Detection

Initial methods of detecting objects from video footage utilized handcrafted feature descriptors and classic machine learning classifiers. Dalal and Triggs (2005) [4] introduced the Histogram of Oriented Gradients (HOG), which combined with Support Vector Machines (SVM) became the standard pipeline for such tasks. While computationally modest, these pipelines struggled with inconsistencies that come with video footage such as illumination variabilities, partial object occlusions and pose variations. These limitations would be exaggerated further in our application on football broadcast footage, due to the persistent occlusion of the ball or players by other players, pitch elements or the background.

Convolutional Neural Networks (CNNs) however demonstrated that learning

R-CNN: Regions with CNN features

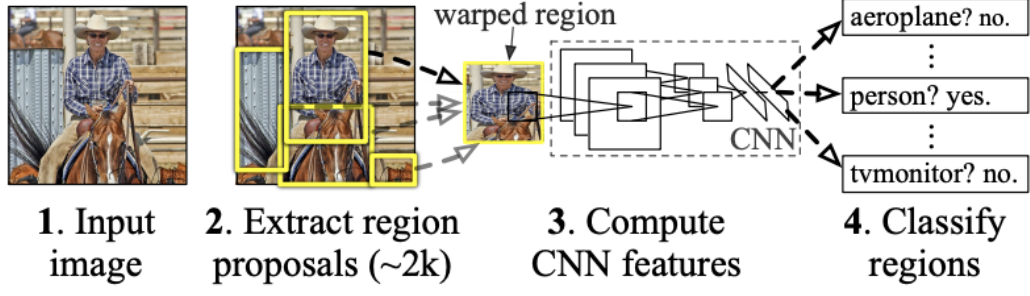


Figure 2.1. System overview of early R-CNN object detection (Girshick et al., 2014 [6])

features from labelled data vastly outperforms handcrafted feature descriptions, as first showcased by Alex Krizhevsky (2012) [9], and subsequent architectures such as VGGNet and ResNet. In the same object detection domain, the Region-based CNN (R-CNN) architecture, developed by Girschick et al. (2014) [6], and refined in later versions into Fast R-CNN and Faster R-CNN, achieved what was for the time state-of-the-art accuracy by combining selective search regions with CNN classifiers. It proved computationally excessive for real-time detection however, with the two-stage pipeline requiring approximately 13 seconds per image on a GPU and 53 seconds per image on a CPU (Girschick et al., 2014) [6].

2.2.2 Single-Stage detection: The YOLO paradigm

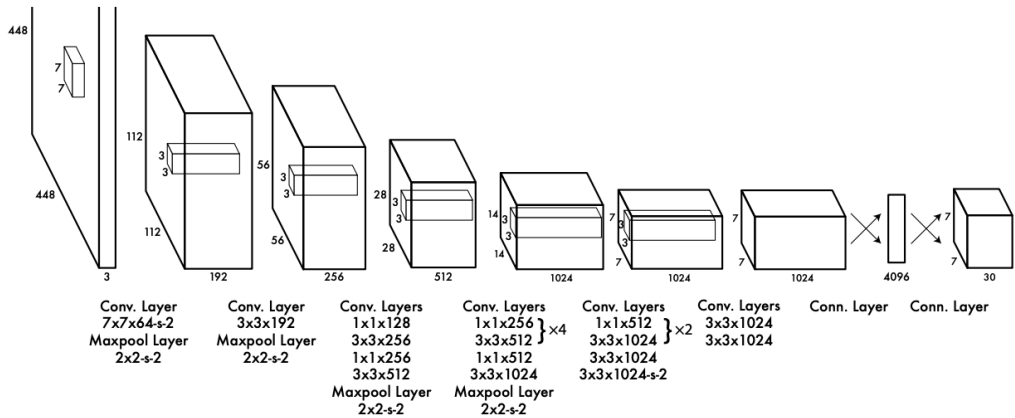


Figure 2.2. Architecture of the first YOLO model (Redmon et al., 2016) [12]

The shift from two-stage detection to a single-stage approach was first introduced by Redmon et al. (2016) [12] in the You Only Look Once (YOLO) publica-

tion. By tackling detection as a single regression problem, the You Only Look Once paradigm processes an entire frame in one forward pass, predicting box boundaries and class probabilities simultaneously. This removed the very costly step of region proposal completely, enabling the original YOLO detector to perform at 45 frames per second on a Titan X GPU. This was a dramatic speed improvement over R-CNN, though at a cost of lower accuracy on small objects.

Three concepts that make the foundations of the architecture evaluated in this thesis were introduced by the YOLO paradigm.

1. The grid based prediction structure makes sure that spatial localization and classification are handled together, removing the computational drawback of a distinct proposal stage seen in earlier approaches.
2. Anchor boxes introduced in later models (Redmon and Farhadi, 2017) [13], supply prior bounding box shapes derived through k-means clustering. This means the model only needs to predict offsets rather than finding absolute coordinates.
3. Multi-scale detection, introduced in YOLOv3 (Redmon and Farhadi, 2018) [14], managed to significantly improve small object detection, a previous limitation of the YOLO architecture, through the use of feature maps at three different resolutions.

2.2.3 Target Performance Metrics

Object detection models are evaluated using two primary standard metrics. Mean Average Precision (mAP) provides a summary of detection quality across all classes and Intersection over Union (IoU) thresholds. The mAP@0.5 metric is computed at an IoU threshold of 0.5. It constituted a simple indication of whether the detector’s produced bounding boxes are substantially overlapping with known ground truth. The mAP@0.5:0.95 metric is a stricter version of mAP@0.5, averaging precision across IoU thresholds from 0.5 to 0.95. It penalizes imprecise localization and provides a more demanding picture of detection accuracy.

Inference speed is defined as latency per image in milliseconds or its reciprocal, frames per second (FPS). For deployment on consumer hardware, both CPU and GPU latency figures are relevant, as the compute capabilities of consumer devices span a wide spectrum from integrated graphics to discrete GPUs.

The trade-off between mAP and inference latency makes up the central analytical domain of this study, and these metric definitions therefore form the evaluative vocabulary upon which the comparative analysis is built.

2.3 Related Work

2.3.1 The Evolution of the YOLO Architecture

Following the original YOLO detector, successive versions introduced targeted improvements that progressively narrowed the accuracy gap between single-stage and two-stage approaches, especially when targeting small objects. The work that gave us YOLOv4 (Bochkovskiy et al., [3]) consolidated a large body of practices, including the CSPDarknet53 backbone, a PANet neck for improved feature pyramid fusion, Mosaic data augmentation, and the CioU (Complete Intersection over Union) loss function, into a unified architecture that set a new state-of-the-art among real-time object detectors on the MS COCO benchmark.

YOLOv8, released by Ultralytics in January 2023 [18], represents the architectural base upon which this thesis is built. It introduced several significant refinements over preceding models. The backbone replaces the C3 (Cross Stage Partial with three convolutions) module with a redesigned Cross Stage Partial (C2f) module that improves feature reuse and gradient propagation through more efficient cross-stage connections and lightweight bottleneck layers. This design improves gradient flow during training and reduces parameter count relative to the previous CSP blocks. The neck retains a PANet-based feature pyramid and retains the Spatial Pyramid Pooling Fast (SPPF) module introduced in latter models of YOLOv5 to efficiently capture multi-scale context. The detection head represents the most consequential architectural change: YOLOv8 adopts an anchor-free, decoupled head that separately predicts bounding box regression and object classification. By eliminating predefined anchor boxes, the model generalizes more readily to custom datasets where object aspect ratios differ from those of the general-purpose COCO training set, a property of particular value when fine-tuning on a domain-specific dataset.

YOLOv8 is offered in five variants, each in a different scale: nano, small, medium, large and extra large. The models are scaled by adjusting the width and depth of the backbone and neck. The scaling is set such as each following step increases in magnitude from the previous in both parameter count and mAP scores. This behaviour makes this architecture suitable for comparative studies as the topology is held constant while only the model capacity differs.

2.3.2 YOLOv8 Variant Comparisons

Previous studies have systematically compared YOLOv8 variants across different hardware configurations and application domains, providing an empirical context

for the present thesis. Alqahtani et al. (2024) [2] conducted a comprehensive benchmarking study of YOLOv8 variants (Nano, Small, and Medium) alongside EfficientDet Lite and SSD MobileNet models, deployed on Raspberry Pi 3, 4, and 5 devices with and without TPU (Tensor Processing Unit) accelerators, as well as on a Jetson Orin Nano. Their findings confirmed that lower-capacity models such as YOLOv8n deliver substantially lower inference latency and energy consumption, while higher-capacity models such as YOLOv8m provide superior mAP at the cost of increased computational overhead. Most importantly, they also identified that the model rank in regard to inference speed is largely dependant on the hardware environment, with different processing units resulting in different efficiency profiles.

Concretely on our football domain, a variant comparison was conducted by Tanapatpiboon et al. (2025) [16] through football player detection and heatmap generation. The authors evaluated YOLOv5m6, YOLOv5l-tph, and related variants on broadcast footage and showed that YOLO object detection can succesfully be utilized in this application. However, they only focused on tracking team positions and ignored important information like the ball position and the distinction of the goalkeeper.

Ahmad Yousef et al. (2025) [1] conducted a rigorous comparison of the YOLOv8x (extra-large) and YOLOv12x models on the task of football analytics. Their results also confirmed the consistent pattern of larger models with more parameters achieving higher accuracy scores while requiring more computational capacity.

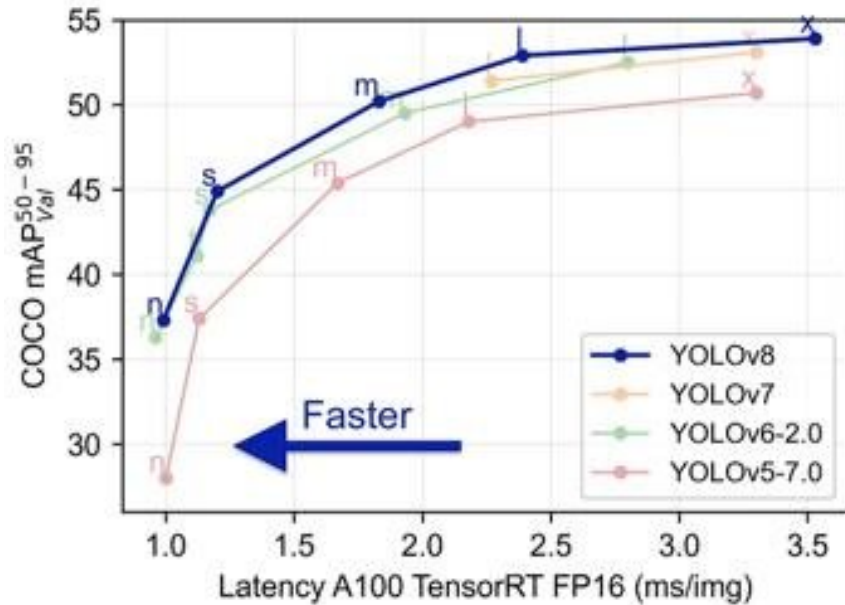


Figure 2.3. A comparison between YOLOv8 and other YOLO models (Ultralytics, 2023) [18]

The aforementioned pattern was also confirmed by Markappa et al. (2024) [10], who conducted a comparison on the different architectures of YOLO models, namely YOLOv3, YOLOv5, YOLOv8, and YOLOv9. This comparison was made on the SoccerNet dataset, and their findings showed interestingly that the YOLOv8 models were able to achieve higher accuracy than the successor, YOLOv9, however this also came at the cost of higher inference latency.

2.3.3 YOLO Based Detection in Football Video Footage

The YOLO architecture has been proven to be very impressive in the football domain. The performance evaluation conducted by Tshiani (2025) [17] provides class-level mAP benchmarks from a YOLOv8s-based pipeline trained and evaluated on a custom dataset combining Bundesliga, NCAA Division I, and UPSL broadcast footage. Tshiani reported mAP@0.5 values of 0.993 for player detection, 0.946 for referees, 0.931 for goalkeepers, and only 0.616 for the ball class, highlighting that across all evaluated classes, ball detection remains the most challenging task, primarily due to its small spatial footprint and frequent occlusion.

The same challenge was found in the study made by Ahmad Yousef et al. (2025) [1]. In their comparison between the different YOLO models, ball detection proved to be the most difficult task, with distinctly lower accuracy scores across the different models. However, a solution was found through the addition of a Kallman Filter to the pipeline, which produced significant improvement in the F1-scores for the ball class.

Another study depicting the same difficulties was made from Hartono et al. (2025) [7]. They used an integrated YOLOv5 pipeline for player tracing, team assignment and speed estimation. As seen in the aforementioned studies, player detection was impressive (this study showed player detection accuracy of 94.8%), however ball detection was very difficult with a precision-recall score of 0.478

2.3.4 YOLOv8 Variant Scaling

A recurring theme in the YOLO benchmarking literature is the hardware dependence of inference performance, which motivates the consumer-hardware framing of the present thesis. According to official Ultralytics (2023) [18] documentation, the YOLOv8n model offers a latency of under 1 ms on a A100 TensorRT GPU. However, these figures are not representative of the inference conditions experienced by a university student, a football academy analyst, or an amateur club coach operating on commodity hardware. Consumer hardware, defined for the purposes of this thesis

as mid-range laptops and desktop systems equipped with budget discrete GPUs and integrated graphics, presents a qualitatively different deployment context.

This is where the different scales of the YOLOv8 family come into use. By gradually increasing the parameters and FLOPs (Floating Point Operations), we can aim to find the most effective model for our hardware and usage.

Model	Size (px)	mAP^{val} 50-95	Speed CPU ONNX (ms)	Speed A100 TensorRT (ms)	Params (M)	FLOPs (B)
YOLOv8n	640	37.3	80.4	0.99	3.2	8.7
YOLOv8s	640	44.9	128.4	1.20	11.2	28.6
YOLOv8m	640	50.2	234.7	1.83	25.9	78.9
YOLOv8l	640	52.9	375.2	2.39	43.7	165.2
YOLOv8x	640	53.9	479.1	3.53	68.2	257.8

Table 2.1. YOLOv8 model variants and their performance metrics (Ultralytics, 2023) [18].

Shuvo et al. (2022) [15] identify several established strategies for deploying deep learning models on resource-constrained devices, including pruning, quantization, knowledge distillation, and compact architecture design. Of these, selecting among pre-scaled model variants (the approach taken in this thesis) requires no modification to the model or training pipeline, making it the most practically accessible option for users without specialized engineering expertise.

2.3.5 The Bundesliga Data Shootout dataset

The Bundesliga Data Shootout dataset, published on Roboflow Universe by Delfos (2022) [5] and derived from the DFL Kaggle competition, provides YOLO-format annotations for the player, referee, goalkeeper, and ball classes. Because of its quality and relevance, it is the most used in modern computer vision studies inside the football context, making it the most contextually relevant publicly available benchmark for this work.

2.4 Positioning of the Present Study - Summary

The review conducted in this chapter traces the development of object detection approaches from rudimentary handcrafted methods like HOG combined with SVMs, progressing into the R-CNN two-stage process, finally maturing into the YOLO paradigm which delivers strong accuracy while achieving real-time video processing. Within that paradigm the current YOLOv8 proves to be the best, achieving impressive accuracy scores and offering different scale models in order to adapt to different tasks and environments that practitioners have.

Its application to football analytics is recent and well established. Markappa et al. (2024) [10] confirm that YOLOv8 consistently outperforms earlier versions on football-specific data, while Tshiani (2025) [17] and Ahmad Yousef et al. (2025) [1] provide the closest empirical comparators to the present work, establishing that player detection approaches saturation across model sizes while ball detection remains the persistent challenge of the domain.

The review also reveals that existing comparisons of these YOLOv8 variants are not fully adapted to the task we are aiming to achieve, which is to implement the YOLOv8 models into consumer grade hardware. From our investigation, these comparisons are either made in datasets unrelated to football footage, which means they miss the specific challenges such a task poses, or they are conducted on very expensive hardware configurations, meaning that the models might not be viable to the target demographic.

The present thesis addresses these questions through a controlled evaluation of YOLOv8n, YOLOv8s, and YOLOv8m on the Bundesliga Data Shootout dataset, conducted specifically on consumer-grade hardware across three different price points, with the aim of providing practical model selection guidance for users operating under realistic resource constraints

CHAPTER 3

METHODOLOGY

3.1 Introduction

This chapter describes in detail the methodology used to conduct a controller comparison of the three lightest variants of the YOLOv8 model: nano (YOLOv8n), small (YOLOv8s), and medium (YOLOv8m). The specific task under which they will be tested is the detection of the four distinct classes of player, referee, goalkeeper and ball. The evaluation consists of two primary dimensions, detection accuracy and inference speed. The environment under which the models will be tested consists of three consumer level hardware configurations, in order to offer a real picture of what a low budget practitioner will experience when using them.

3.2 Research Design

The target variable of this study is the model architecture and scale. As such, measures have to be taken in order to not allow any other factors to influence the results. The three YOLOv8 variants share identical architectural topology and differ only in depth and width scaling coefficients, which in turn determine parameter count and computational cost. This property makes them perfect for a comparative study such as this one, as any differences that get observed would be as an unambiguous result of model capacity and not due to architecture, training pipeline, or data processing.

To give each model the same conditions, all other influencing attributes have to remain identical. This includes the dataset, the way it is preprocessed, training hyperparameters, augmentation configuration and the inference settings during evaluation. Training is conducted once on a single machine equipped with a dedicated GPU. It follows a fine-tuning approach which starts from COCO pretrained weights then learns the domain-related details from our dataset. Inference benchmarking is conducted on each of the three devices using the same trained weights and test video. From this we can conclude that all latency differences will be purely as a result of the interaction between model and hardware.

3.3 Dataset

3.3.1 Source

The dataset used in this study is the Bundesliga Data Shootout dataset, published by Delfos (2022) [5] and hosted on the Roboflow Universe platform. It was downloaded via the Roboflow Python API, ensuring reproducibility across machines. The dataset consists of annotated frames extracted from broadcast footage of Bundesliga football matches, making it particularly relevant to the detection task this thesis studies. It was selected because it provides YOLO-format annotations compatible with the Ultralytics pipeline, covers all four object classes of interest (players, goalkeepers, referees, and the ball), and is publicly versioned, enabling easy future replication. For our testing, however, we trained our models on the two most relevant classes, the player and ball class, in order to get the most direct result on what coaches and scouts might need when evaluating talent.

3.3.2 Dataset Splitting

The dataset version downloaded from Roboflow includes a predefined train, validation, and test split. Using the predefined split, rather than applying a random split at training time, serves two purposes. First, it ensures that the same images appear in the same partitions across all three models, so that accuracy differences cannot be attributed to variation in which images each model saw during training. Second, it enables direct comparison with future studies that use the same dataset version, as the evaluation set is fixed and consistent.

3.3.3 Data Augmentation

Augmentation for this study was applied in two distinct stages: offline augmentation performed by Roboflow at dataset export time, and online augmentation applied by the YOLOv8 training pipeline during each epoch. Understanding both stages is necessary for an accurate account of the data the models were trained on.

The Roboflow export configuration applied the following augmentations to the base dataset, generating three augmented outputs per original training image and thereby tripling the effective training set size: horizontal and vertical flipping, clockwise and counterclockwise 90° rotations, rotation between -15° and +15°, hue jitter between -20° and +20°, saturation jitter between -25% and +25%, exposure jitter between -10%

and $+10^\circ$, and Mosaic compositing. Several of these augmentations are well-suited to broadcast football footage. Horizontal flipping is appropriate given the bilateral symmetry of the pitch. HSV-space jitter addresses variation in broadcast lighting conditions and camera white balance. Mosaic augmentation, which composites four images into a single training sample, is particularly beneficial for small-object detection and increases the frequency with which the ball appears at varied scales and positions within a single training image.

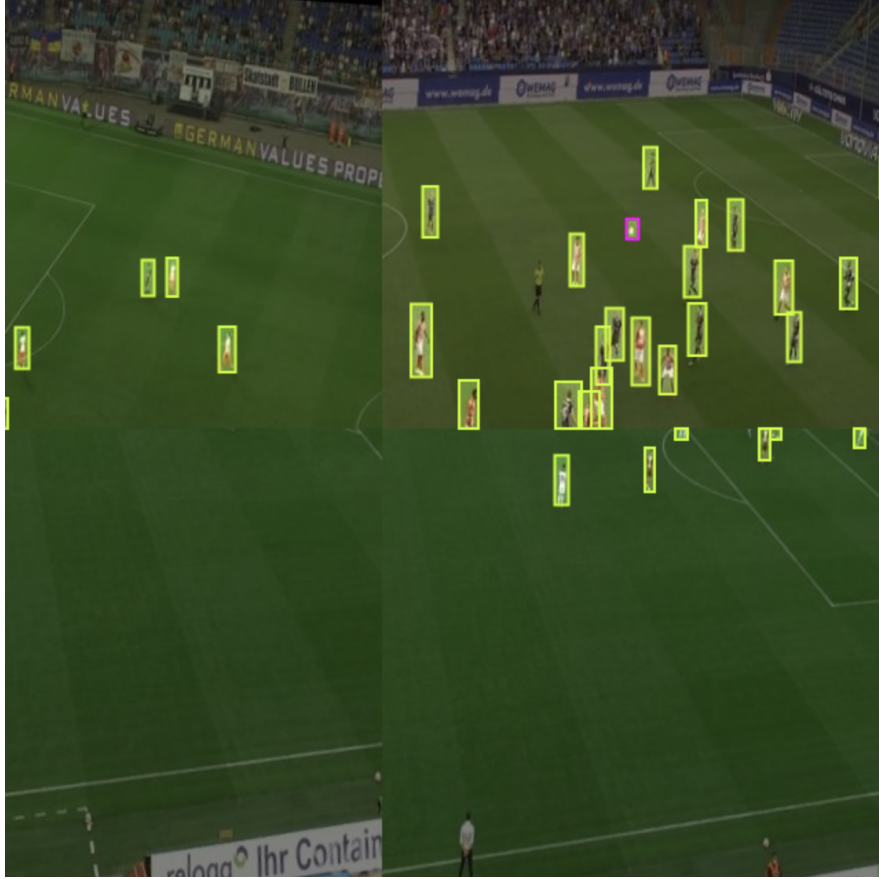


Figure 3.1. An example of an annotated and augmented image in our dataset [5]

The augmentations are not perfect, however, and make up some of the limitations of this study. Vertical flipping and 90° rotations produce frames that are sure to never show up on regular usage input, as football broadcast footage is always oriented horizontally relative to the pitch.

The online augmentation performed applied by the YOLO training pipeline has been left at default. The reasoning for this is that the aim of this study is not to create the absolute best model, rather to focus more on the accuracy-speed balance on different hardware configurations.

Augmentation	Setting
Horizontal flip	Applied
Vertical flip	Applied
90° rotation (CW and CCW)	Applied
Rotation	$\pm 15^\circ$
Hue jitter	$\pm 20^\circ$
Saturation jitter	$\pm 25\%$
Exposure jitter	$\pm 10\%$
Mosaic	Applied
Outputs per image	$3\times$

Table 3.1. Augmentation configuration applied to the dataset from default Roboflow configurations.

3.4 Training Procedure

Due to the relatively small size of our dataset, a transfer learning strategy is implemented with the goal to avoid high variance results. The COCO dataset offers low-level visual features which are shared with broadcast footage. This in turn allows the model to rapidly adapt to class distributions specific to our domain.

The key requirement for a valid comparative evaluation is that all hyperparameters remain equal for all model variants. To achieve this, a single configuration dictionary is defined and shared to all models in their training call. The full configuration is described in table 3.2.

A physical batch size of 8 is used for all three variants to fit within the 12 GB VRAM of the training GPU for the largest evaluated model (YOLOv8m). The nominal batch size parameter (nbs) is set to 64, matching the YOLOv8 default, so that the learning rate for all three models is computed as if training with a batch of 64. This ensures identical effective learning rate dynamics across variants. A fixed random seed of 42 and deterministic mode are applied to guarantee reproducibility across runs.

3.5 Hardware Configurations

The central contribution of this thesis lies on the evaluation of inference performances across different hardware configurations, all at the consumer level. These devices are described below.

Device A is a desktop computer equipped with a discrete GPU, the NVIDIA GeForce RTX 3060 with 12 GB of GDDR6 memory. This machine takes on the

Parameter	Value	Justification
epochs	50	Sufficient for convergence; early stopping prevents overfitting
imgsz	640	YOLOv8 standard input resolution
optimizer	SGD	Default YOLOv8 optimizer; identical dynamics across variants
lr0	0.01	Default YOLOv8 learning rate; explicitly locked
nbs	64	Nominal batch size; ensures identical effective LR scaling across variants
batch	8	Physical batch size fitting RTX 3060 12 GB VRAM for all variants
amp	False	Full FP32; prevents hardware-specific numeric differences
seed	42	Fixed seed for reproducible weight initialization and data ordering
deterministic	True	Disables non-deterministic CUDA ops for reproducibility
patience	20	Proportionate to 50-epoch budget; allows recovery from plateaus

Table 3.2. Shared training configuration applied identically to all three YOLOv8 variants.

training of the models, and also represents a discrete GPU baseline as a low end offering in the market, targeted towards gaming and low level workstation builds. Inference on this device runs with CUDA acceleration, making it the fastest of the three configurations and the appropriate upper-bound reference for the speed evaluation.

Device B is an Apple MacBook Pro equipped with a base Apple M4 system-on-chip (SOC) and 16GB of unified memory. Through the Apple Metal Performance Shaders (MPS) backend, PyTorch accesses the M4 GPU. The unique memory architecture and MPS execution path offer a distinct environment compared to both the CUDA-based GPU and the CPU only laptop, giving us the ability to evaluate a wide range of hardware configurations.

Device C is a Lenovo ThinkPad T490s equipped with an Intel Core i7-8665U CPU and no discrete GPU. Since the inference on this machine is run entirely on the CPU with no hardware acceleration, it represents the lower bound reference for our deployment scenarios. It also offers a very demanding environment for the YOLO models to perform under the absence of GPU acceleration.

	Device A	Device B	Device C
Type	Desktop PC	Laptop	Laptop
CPU	AMD Ryzen 5 5600X	Apple M4	Intel Core i7-8665U
RAM	16 GB DDR4	16 GB Unified	16 GB DDR4
GPU / Accelerator	NVIDIA RTX 3060 12 GB	Apple M4 GPU (MPS)	None (CPU only)
PyTorch Backend	CUDA	MPS	CPU
OS	Windows 10	macOS	Windows 10

Table 3.3. Hardware specifications of the three consumer-grade devices used for inference benchmarking.

3.6 Inference Benchmarking

3.6.1 Settings

All inference settings are held constant across devices and model variants. The confidence threshold is set at a fixed value, the NMS IoU threshold is set to 0.45, the input image resolution is 640×640 , and the maximum number of detections per frame is capped at 300. These settings are specified explicitly in the inference configuration rather than relying on model defaults, for the same reason that training hyperparameters are explicitly specified: to ensure that the comparison is not confounded by implicit parameter differences between runs.

3.6.2 Latency Measurement

Inference latency is measured as time per frame, recorded using Python’s time module around the model’s forward pass for each frame of the test video. The first several frames are treated as warmup and excluded from the latency statistics to avoid penalizing the model for compilation or launch overheads that occurs only at the start of inference. Mean latency is computed over all remaining frames. Frames per second is derived as 1000 divided by mean latency in milliseconds.

3.7 Summary

This chapter has presented the methodology for a controlled comparative evaluation of YOLOv8n, YOLOv8s, and YOLOv8m on object detection from broadcast

football footage. The study is grounded in a strict experimental design in which model scale is the sole independent variable, with all other training and evaluation conditions held constant. The Bundesliga Data Shootout dataset provides the annotated football footage used for training and accuracy evaluation. The three model variants are fine-tuned from COCO-pretrained weights using an identical hyperparameter configuration. Inference speed is evaluated across three consumer-grade hardware configurations using a fixed test video and consistent inference settings. The following chapter presents the results of the training and inference experiments.

CHAPTER 4

RESULTS AND DISCUSSION

4.1 Training Results

This section will present the training results for all three YOLOv8 variants, trained on the Bundesliga Shootout Dataset for player and ball detection. Each model was trained using the identical hyperparameters described on the previous chapter. Training was set for 50 epochs, but early-stopping was implemented to prevent any models from training longer than necessary. The best performing checkpoint per model is used for all reported metrics, saved at the epoch with the highest validation mAP@0.5.

4.1.1 Training Convergence

Figure 4.1 depicts a clear picture of the evolution of training box loss, validation mAP@0.5, precision, and recall across 50 epochs for each model. Validation loss tracked training loss throughout the epochs, meaning we do not see any signs of overfitting. Training box loss decreased from approximately 2.3 in epoch 1 to below 1.8 for all models by epoch 50, with YOLOv8m achieving the lowest final training loss (1.48) due to its greater model capacity.

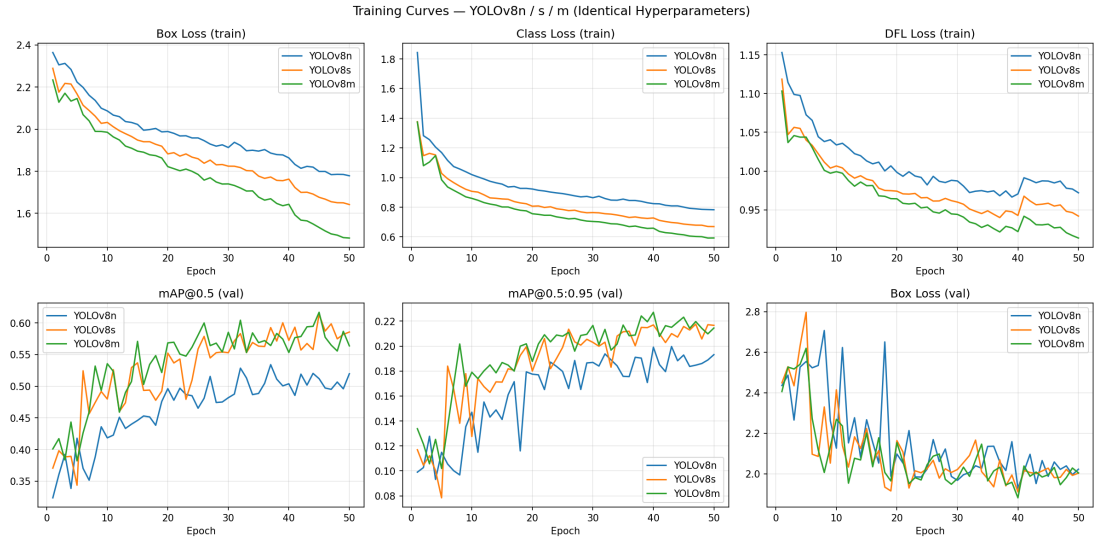


Figure 4.1. Training curves for all three models during 50 epochs

The mAP@0.5 curves show that initial improvement for all three models was rapid, especially during the first 10 epochs. The best performance was achieved around the 40th epoch for all models, after which we notice plateaus or minor fluctuations. This pattern is usual for small datasets due to the model exhausting learning material relatively quickly.

4.1.2 Detection Metrics

Table 4.1 reports the validation metrics for each model’s best epoch. The small and medium models achieve almost identical overall mAP@0.5 scores of 0.613 and 0.617 respectively, as the difference of 0.004 is not statistically meaningful in a 38 image validation set. The nano model performs noticeably worse in all metrics, clearly showing the limitations of its smaller capacity of 3M parameters for this detection task.

Model	Precision	Recall	mAP@0.5	mAP@0.5:0.95	Train Time (min)
YOLOv8n	0.736	0.528	0.534	0.191	11.8
YOLOv8s	0.775	0.613	0.613	0.216	33.5
YOLOv8m	0.698	0.632	0.617	0.223	50.4

Table 4.1. Best-epoch validation metrics for all three model variants

An interesting distinction between the YOLOv8s and YOLOv8m medium models lies on the precision-recall tradeoff. The small model achieves higher precision,

while medium model achieves higher recall. This indicates that the medium model is more aggressive with its detections and therefore generates more true positives. The small model is pickier and better calibrated for the size of this dataset.

4.1.3 Class Detection Performance

Breaking down the class detection performance reveals a very important finding in our training evaluation. From the figures in Table 4.2 we can see that while player detection is strong across all models, the ball detection remains substantially weaker. This weakness lines up with other studies of the same domain which we covered in the second chapter of this thesis. This weakness, however, also proves to be the biggest difference maker across model sizes.

Model	Ball AP	Player AP
YOLOv8n	0.097	0.906
YOLOv8s	0.270	0.927
YOLOv8m	0.167	0.940

Table 4.2. Per-class average precision (AP) for all three model variants

The YOLOv8n model continues to show the limitations of its small parameter size, with the lowest metrics across both classes. Between the small and medium models, we notice more interesting differences. While the medium model performs slightly better at player detection, the small model shows distinctly better ball detection performance. This non-linear relationship between model size and detection accuracy shows a weakness of our study, which is the dataset size. With the small number of ball instances in the validation set, the larger model struggles to generalise to this underrepresented class.

4.1.4 Precision-Recall Curves

Below are the resulting precision-recall curves for each of our trained models. In all cases, a similar picture as seen above shows. The player detection curve shows strong performance by occupying the upper-right region of the plot, while the ball curve is substantially lower and more irregular.

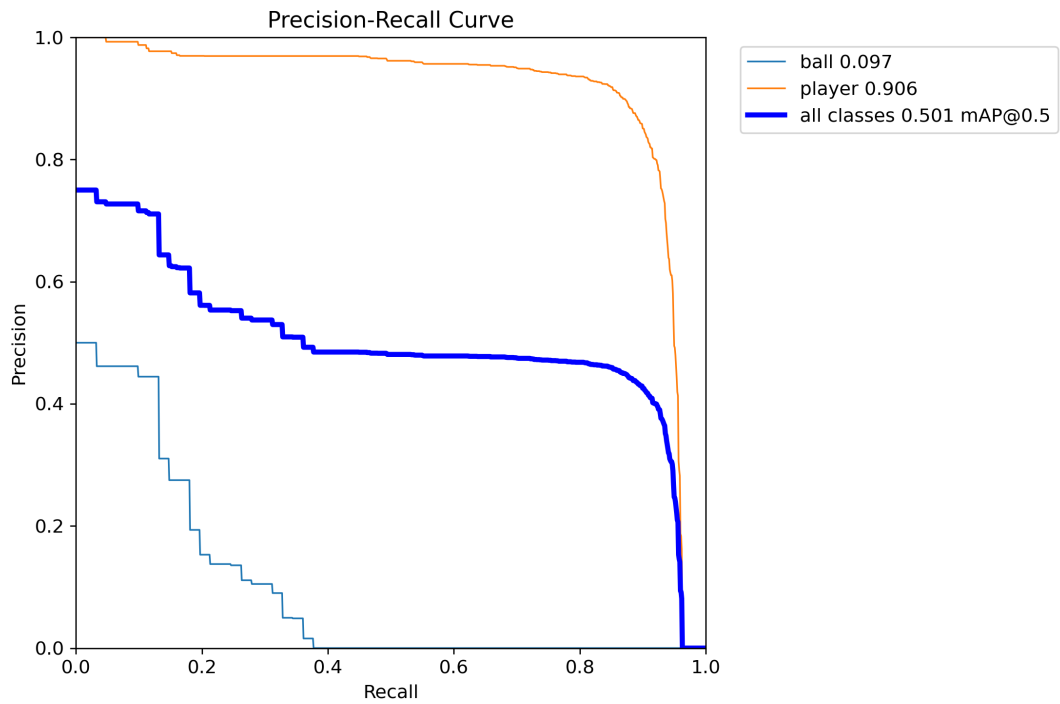


Figure 4.2. Precision-Recall curve for the YOLOv8n model

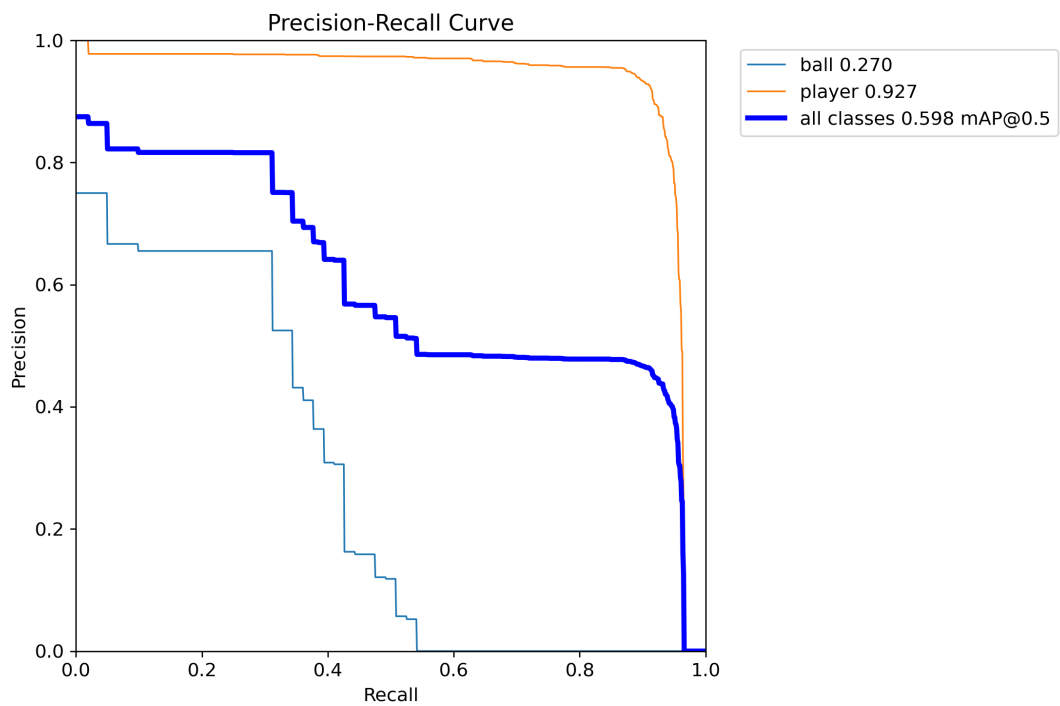


Figure 4.3. Precision-Recall curve for the YOLOv8s model

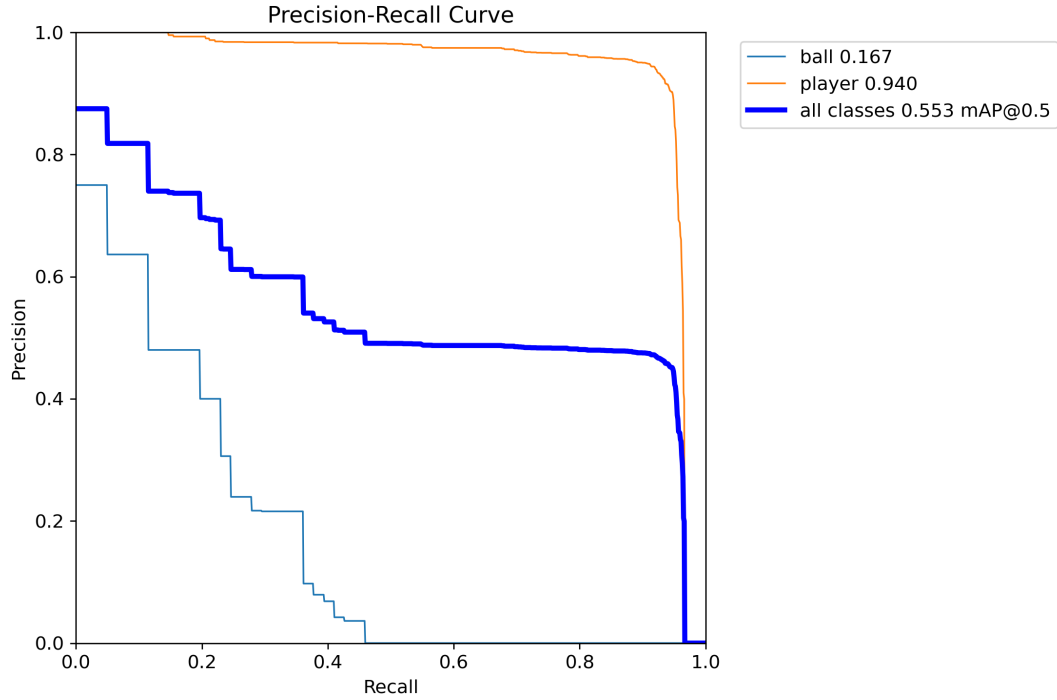


Figure 4.4. Precision-Recall curve for the YOLOv8m model

4.1.5 Confusion Matrices

Figures 4.5, 4.6 and 4.7 present the confusion matrices for each model evaluated on the validation set. These results reinforce the class-level AP findings. Player detections are accurate across all models. A significant error pattern is found from player detections being assigned to the background class. This shows missed detection rather than confusion between object classes.

Ball detections remain, as seen earlier, particularly difficult. The YOLOv8 model performs especially poorly, with only 11 true ball instances being correctly detected and 48 missed as background. This comes as a limitation of the image resolution picked for our pipeline, as a 640x640 image proves to be too blurry for small objects such as the ball to be properly differentiated from the background. YOLOv8s improves on the nano model to 22 correct detections and 39 missed, representing the best ball recall of the three models.

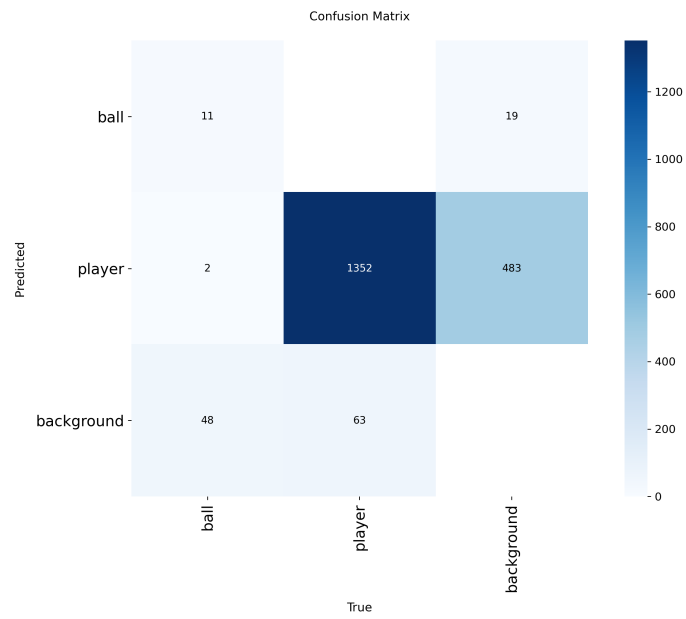


Figure 4.5. Confusion matrix for the YOLOv8n model

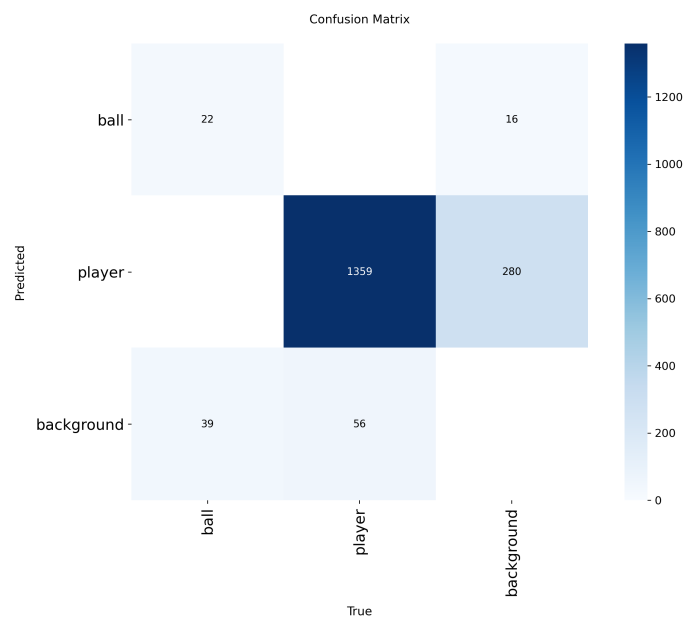


Figure 4.6. Confusion matrix for the YOLOv8s model

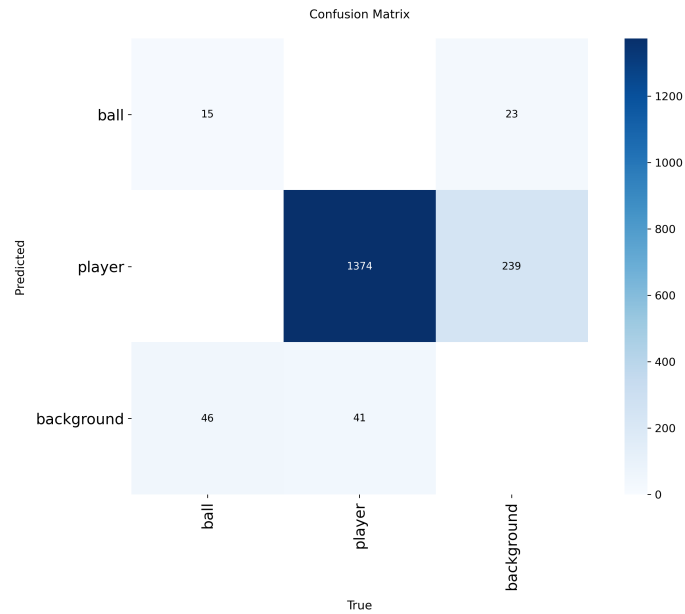


Figure 4.7. Confusion matrix for the YOLOv8m model

4.1.6 Training Summary

The training results demonstrate a very clear picture of the accuracy of each model, as well as their shortcomings. Player detection is very strong across all three variants, with each of them achieving $mAP@0.5$ scores of over 0.9 on that class. However, as previous studies investigated in our literature review section of this thesis, ball detection remains a challenge, with none of the models achieving $mAP@0.5$ scores over 0.3. This indicates the dataset size and image resolution as constraints, as there are not enough instances of the ball class and the ball being a small object gets confused with the background due to the small number of pixels.

4.2 Inference Benchmarks

This section will present the inference benchmark results for all three YOLOv8 variants, in each of our hardware configurations as described in the methodology chapter of this thesis. The inference test video is the same for all: a 1920x1080 video which is 15 seconds long with 30 FPS. This gives us a total of 750 frames, which get resized to 640x640 to fit in our pipeline.

4.2.1 Device A – Discrete NVIDIA GeForce RTX 3060 12GB GPU (CUDA)

On Device A the three models deliver markedly different throughput. Interestingly the nano variant performed very slightly slower than the small, but that can be attributed to warmups as the nano inference was run first. On equal conditions, the nano variant should be expected to perform faster. Another significant gap is between the small and medium variants, as we notice a drop of 33% in FPS going from YOLOv8s to YOLOv8m.

Model	Total dets.	Dets/frame	Mean conf.	Med. conf.	Std conf.	Inference FPS
YOLOv8n	15,518	20.69	0.503	0.520	0.109	78.3
YOLOv8s	17,114	22.82	0.563	0.591	0.119	82.3
YOLOv8m	16,686	22.25	0.578	0.609	0.111	55.1

Table 4.3. Inference summary statistics for all three YOLOv8 variants on Device A (RTX 3060, CUDA).

Related to the overall performance of the models, all of them comfortably exceed the 25 FPS threshold commonly considered the minimum for smooth real-time video. This shows that a machine fitted with a discrete GPU should have no problem on object detection from video, and in fact may even be encouraged to try the larger YOLOv8l(large) model.

4.2.2 Detection Volume and Confidence

YOLOv8s produces the highest total detection count at 17,114 detections across 750 frames (22.82 detections per frame on average), compared to 16,686 for YOLOv8m and 15,518 for YOLOv8n. Importantly, no frame across any of the three models returned zero detections, indicating consistent coverage of the video content under the selected confidence threshold.

The confidence statistics reveal a clear ordering with model scale. Mean confidence increases from 0.503 (YOLOv8n) to 0.563 (YOLOv8s) to 0.578 (YOLOv8m), as does median confidence. This pattern reflects the expected relationship between model capacity and calibration.

The per-class breakdown in Table 4.4 reinforces the class-level findings from the training evaluation. Player detections dominate the detection volume across all three models, accounting for between 95.8% and 97.9% of all detections. Ball detec-

tions remain a small fraction of the total in all cases, consistent with the low ball AP scores reported in Section 4.1.3.

Model	Dets [ball]	Dets [player]	Ball %	Player %	Mean conf.
YOLOv8n	319	15,199	2.1%	97.9%	0.503
YOLOv8s	717	16,397	4.2%	95.8%	0.563
YOLOv8m	529	16,157	3.2%	96.8%	0.578

Table 4.4. Per-class detection counts and proportions for all three YOLOv8 variants on Device A (RTX 3060, CUDA).

Table 4.5 and the subsequent Figure 4.8 present the full confidence score distribution across all detections for each model. The distributions show a clear shift toward higher confidence bins as model capacity increases. YOLOv8n is heavily concentrated in the 0.40–0.60 range, with its modal bin at 0.50–0.60 (5,583 detections) and only 86 detections above 0.70. YOLOv8s and YOLOv8m both show a marked redistribution of detections into the 0.60–0.70 bin, which becomes the modal bin for both models (6,638 and 7,833 detections respectively), with YOLOv8s also contributing 1,381 detections in the 0.70–0.80 range. No detections from any model exceed 0.80, which reflects the inherent difficulty of the task, particularly for ball detection, and is consistent with the moderate mAP@0.5 scores reported during training.

Confidence bin	YOLOv8n	YOLOv8s	YOLOv8m
0.25 – 0.30	801	689	520
0.30 – 0.40	2,247	1,462	1,125
0.40 – 0.50	3,680	2,253	1,824
0.50 – 0.60	5,583	4,691	4,299
0.60 – 0.70	3,121	6,638	7,833
0.70 – 0.80	86	1,381	1,085
0.80 – 0.90	0	0	0
0.90 – 1.00	0	0	0

Table 4.5. Confidence score distribution across bins for all three YOLOv8 variants on Device A (RTX 3060, CUDA).

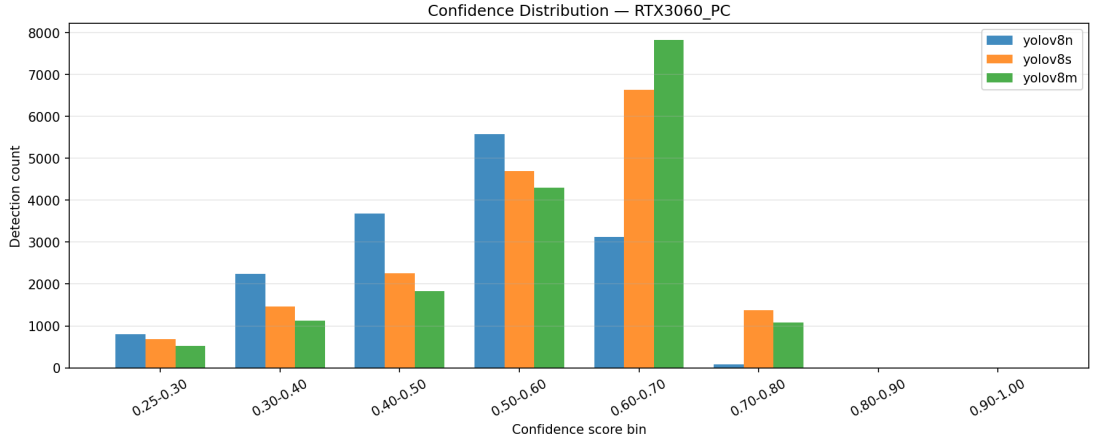


Figure 4.8. Confidence distribution for all models on Device A(RTX 3060, CUDA).

4.2.3 Device B – Apple MacBook Pro M4 (MPS)

On Device B the three models show a noticeably different speed profile compared to Device A. YOLOv8n and YOLOv8s run at nearly identical throughput of 57.1 and 56.5 FPS respectively, a gap of only 1.1% that is not practically meaningful. The medium variant drops more sharply to 34.6 FPS, a reduction of 38.8% relative to YOLOv8s. Compared to Device A, all three models are slower: YOLOv8n loses 27.1% of its RTX 3060 throughput, YOLOv8s loses 31.3%, and YOLOv8m loses 37.2%, indicating that the performance gap widens with model size on this hardware. Despite this, all three variants remain well above the 25 FPS threshold considered the minimum for smooth real-time video processing.

Model	Total dets.	Dets/frame	Mean conf.	Med. conf.	Std conf.	Inference FPS
YOLOv8n	15,526	20.70	0.503	0.520	0.109	57.1
YOLOv8s	17,094	22.79	0.564	0.591	0.119	56.5
YOLOv8m	16,691	22.25	0.577	0.609	0.111	34.6

Table 4.6. Inference summary statistics for all three YOLOv8 variants on Device B (Apple M4, MPS).

The detection counts on Device B are virtually identical to those recorded on Device A, which is expected given that the same model weights and inference settings were used across both devices. YOLOv8s again produces the highest total at 17,094 detections (22.79 per frame), followed by YOLOv8m at 16,691 and YOLOv8n at 15,526. No frame returned zero detections across any model, confirming consistent coverage. The confidence statistics are likewise stable: mean confidence follows

the same ordering of 0.503 (YOLOv8n), 0.564 (YOLOv8s), and 0.577 (YOLOv8m), reinforcing that the MPS backend introduces no meaningful change in detection quality relative to CUDA.

The per-class breakdown in Table 4.7 again shows player detections predominantly between 95.8% and 97.9% of total detections. Ball detection counts closely mirror the Device A results, with YOLOv8s producing the highest ball count at 716, followed by YOLOv8m at 529 and YOLOv8n at 322, preserving the same non-linear ordering observed during training.

Model	Dets [ball]	Dets [player]	Ball %	Player %	Mean conf.
YOLOv8n	322	15,204	2.1%	97.9%	0.503
YOLOv8s	716	16,378	4.2%	95.8%	0.564
YOLOv8m	529	16,162	3.2%	96.8%	0.577

Table 4.7. Per-class detection counts and proportions for all three YOLOv8 variants on Device B (Apple M4, MPS).

Table 4.8 and Figure 4.9 present the confidence score distribution across all detections on Device B. The pattern is near-identical to that observed on Device A, which again confirms that backend differences between CUDA and MPS do not affect detection quality. YOLOv8n remains concentrated in the 0.40–0.60 range, with its modal bin at 0.50–0.60 (5,581 detections) and only 85 detections above 0.70. YOLOv8s and YOLOv8m again shift their modal bin to 0.60–0.70 (6,660 and 7,831 detections respectively), and YOLOv8s contributes 1,382 detections in the 0.70–0.80 range. As on Device A, no detections from any model exceed 0.80.

Confidence bin	YOLOv8n	YOLOv8s	YOLOv8m
0.25 – 0.30	799	673	535
0.30 – 0.40	2,235	1,474	1,121
0.40 – 0.50	3,710	2,254	1,792
0.50 – 0.60	5,581	4,651	4,343
0.60 – 0.70	3,116	6,660	7,831
0.70 – 0.80	85	1,382	1,069
0.80 – 0.90	0	0	0
0.90 – 1.00	0	0	0

Table 4.8. Confidence score distribution across bins for all three YOLOv8 variants on Device B (Apple M4, MPS).

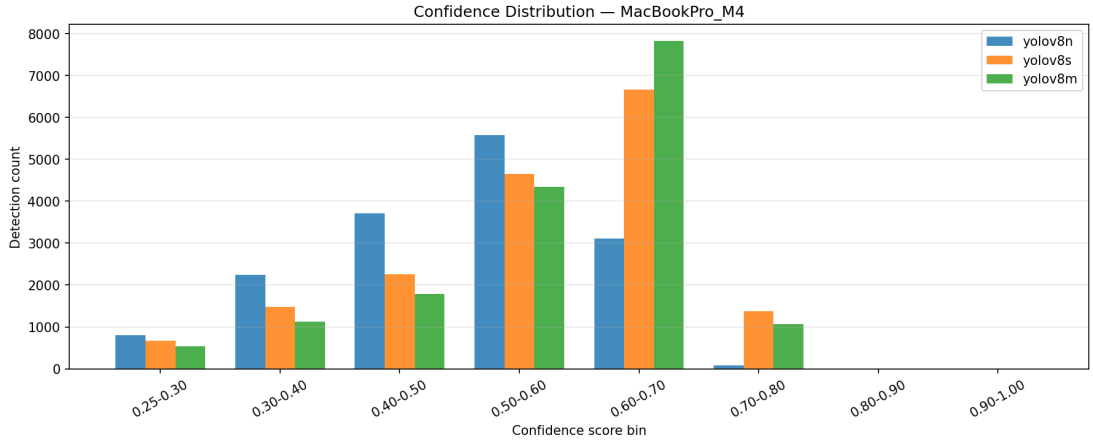


Figure 4.9. Confidence distribution for all models on Device B(MacBook Pro M4, MPS).

4.2.4 Device C – Lenovo ThinkPad T490s (CPU only)

Device C presents the most difficult environment out of the three in our testing. Due to the lack of a GPU, inference is run entirely on the Intel Core i7-8665U CPU with no hardware acceleration. As a consequence of this, the impact on performance is dramatic. Only the nano model reaches double digits in FPS figures at 14.1 FPS, while the others are far below (YOLOv8s – 6.6 FPS, YOLOv8m – 2.9 FPS). As none of the models passes the 25 FPS threshold, Device C proves incapable of providing smooth real-time object detection, even on the lightest and most inaccurate model. This, however, does not mean it does not have its uses, as we will discuss in latter

sections of our thesis.

Model	Total dets.	Dets/frame	Mean conf.	Med. conf.	Std conf.	Inference FPS
YOLOv8n	15,518	20.69	0.503	0.520	0.109	14.1
YOLOv8s	17,110	22.81	0.563	0.591	0.119	6.6
YOLOv8m	16,687	22.25	0.578	0.609	0.111	2.9

Table 4.9. Inference summary statistics for all three YOLOv8 variants on Device C (Lenovo ThinkPad T490s, CPU only).

The per-class breakdown in Table 4.10 again shows the same pattern seen across all devices. Player detections account for the overwhelming majority of all detections, as is to be expected. Ball detection counts are identical to those on Device A, with YOLOv8s producing 717 ball detections, YOLOv8m producing 529, and YOLOv8n producing 319, preserving the non-monotonic ordering established in training across all three hardware configurations.

Table 4.10. Per-class detection counts and proportions for all three YOLOv8 variants on Device C (Lenovo ThinkPad T490s, CPU only).

Model	Dets [ball]	Dets [player]	Ball %	Player %	Mean conf.
YOLOv8n	319	15,199	2.1%	97.9%	0.503
YOLOv8s	717	16,393	4.2%	95.8%	0.563
YOLOv8m	529	16,158	3.2%	96.8%	0.578

Table 4.11 presents the confidence score distribution on Device C. The distributions are near-identical to those recorded on Devices A and B, providing the strongest confirmation yet that the choice of hardware backend, be it CUDA, MPS, or CPU, has no effect on detection confidence. YOLOv8n remains concentrated in the 0.40–0.60 range, with its modal bin at 0.50–0.60 (5,583 detections) and only 86 detections above 0.70. YOLOv8s and YOLOv8m again shift their modal bin to 0.60–0.70 (6,635 and 7,836 detections respectively), with YOLOv8s contributing 1,384 detections in the 0.70–0.80 range. No detections from any model exceed 0.80.

Confidence bin	YOLOv8n	YOLOv8s	YOLOv8m
0.25 – 0.30	800	685	520
0.30 – 0.40	2,247	1,462	1,128
0.40 – 0.50	3,679	2,251	1,821
0.50 – 0.60	5,583	4,693	4,297
0.60 – 0.70	3,123	6,635	7,836
0.70 – 0.80	86	1,384	1,085
0.80 – 0.90	0	0	0
0.90 – 1.00	0	0	0

Table 4.11. Confidence score distribution across bins for all three YOLOv8 variants on Device C (Lenovo ThinkPad T490s, CPU only).

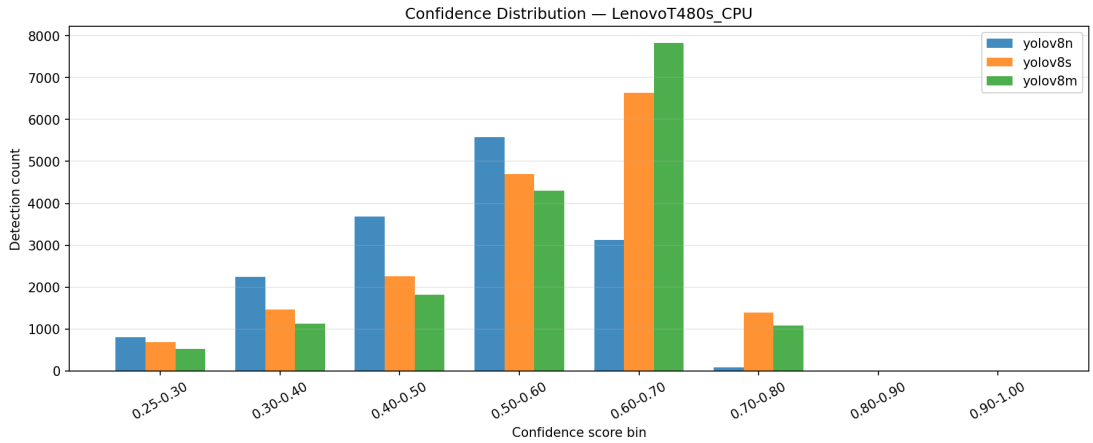


Figure 4.10. Confidence distribution for all models on Device C(Lenovo Thinkpad T490s, CPU).

4.3 Summary

This chapter presented the results of the training and inference experiments across all three YOLOv8 variants and the three consumer-grade hardware configurations. The training evaluation established a clear accuracy hierarchy: YOLOv8m achieves the highest overall mAP@0.5 at 0.617, followed closely by YOLOv8s at 0.613, while YOLOv8n trails noticeably at 0.534. Player detection is strong across all three models, with each exceeding an AP of 0.90 on that class. Ball detection remains the persistent challenge of the domain, with no model surpassing an AP

of 0.27, a limitation attributed to the small spatial footprint of the ball at 640×640 resolution and the limited number of ball instances in the dataset. Notably, YOLOv8s outperforms YOLOv8m on ball detection despite being the smaller model, a non-linear result explained by the medium model’s tendency to overfit to the underrepresented ball class.

The inference benchmarks reveal that hardware selection has a decisive impact on throughput but no effect on detection quality, as confidence distributions and detection counts were consistent across all three devices. On Device A, all three models run at real-time speeds under CUDA, with YOLOv8n and YOLOv8s both exceeding 78 FPS and YOLOv8m reaching 55 FPS. Device B delivers moderate throughput under MPS, with YOLOv8n and YOLOv8s both near 57 FPS and YOLOv8m at 34.6 FPS, all above the 25 FPS real-time threshold. Device C exposes the severe cost of CPU-only inference: only YOLOv8n reaches 14.1 FPS, while YOLOv8s and YOLOv8m fall to 6.6 and 2.9 FPS respectively, making them unsuitable for real-time deployment on this hardware tier.

CHAPTER 5

CONCLUSION AND FUTURE WORK

5.1 Conclusion

The goal of this thesis was to evaluate the accuracy-speed balance of the three lightest variants of the YOLOv8 architecture (nano, small and medium). The task on which they were compared consisted of football player and ball detection from broadcast video footage, and specifically how these models perform on consumer level hardware. Each model was trained identically on the Bundesliga Data Shootout dataset. Through running an equal inference test on all three machines, we get a clear and coherent picture of how model size and hardware capacity interact. From the training validation results we see a clear accuracy hierarchy. YOLOv8m achieved the highest overall mAP@0.5 at 0.617, followed closely by YOLOv8s at 0.613, while YOLOv8n trailed noticeably at 0.534. Player detection was very effective on all models with each one of them exceeding AP scores of 0.90 on that class. However, as seen in relevant literature covered in Chapter 2, ball detection proved significantly more difficult, with no model surpassing an AP of 0.27.

The inference benchmarks demonstrate that hardware selection has a quintessential impact on throughput while being irrelevant to the accuracy, as confidence distributions and detection counts were consistent across all three devices. On Devices A (RTX 3060, CUDA) and B (Apple M4, MPS), all models tested passed the 25 FPS threshold for smooth real-time video processing, while Device C failed to reach it with even the lightest nano model. Below we give our recommendations for each specific hardware configuration:

- Device A performed especially well, with even the YOLOv8m model performing comfortably above the threshold at 55.1 FPS. In this case, we would even suggest going to the even bigger YOLOv8l(large) model.
- Device B also performed well, but the medium model was only slightly above the threshold at 34.6 FPS. As such, it is the largest model we would suggest.
- Device C made up the most resource-constrained environment in this evaluation, and the results show it. None of the models managed to pass the real-time threshold of 25 FPS, making such a task unfeasible in this hardware configuration.

ration. However, tasks that do not need smooth real-time video processing are still possible, as the nano model performed at 14.1 FPS, which is respectable for the environment.

5.2 Limitations

Several limitations of this study should be acknowledged:

- **Dataset size:** the Bundesliga Data Shootout dataset is relatively small, which contributed to high variance in ball detection metrics and limited the medium model's ability to generalise to the underrepresented ball class.
- **Input resolution:** the standard 640×640 pipeline resolution proved insufficient for reliable ball detection, as the ball occupies very few pixels at this scale and becomes difficult to distinguish from the background.
- **Augmentation imperfections:** the offline augmentation applied by Roboflow included vertical flipping and 90° rotations, which produce orientations that never occur in real broadcast footage and may have introduced noise into the training distribution.
- **Hardware coverage:** the three tested devices, while representative of different budget tiers, do not cover the full range of consumer hardware. Mid-range discrete GPUs, integrated graphics, and ARM-based Windows devices remain outside the scope of this benchmark.

5.3 Limitations

Despite the advancements described in this work, some limitations remain:

- **Texture Dependency:** Like all defocus-based methods, "VoluFusion" struggles with untextured or perfectly reflective surfaces where focus cues are absent.
- **Optical Calibration:** The accuracy of the volumetric projection is highly sensitive to the precision of the intrinsic and extrinsic camera parameters.
- **Ambient Lighting:** Performance may degrade in extremely dynamic lighting conditions where local focus measures become inconsistent.

5.4 Future Work

The most immediate improvements are in the two major limitations identified above. Increasing the input resolution to 1280×1280 would increase the spatial footprint of the ball in each frame, and is expected to improve ball detection AP meaningfully without changing the model architecture. Expanding the training dataset (e.g. by using additional Bundesliga Data Shootout splits or other complementary publicly available football datasets) would reduce the variance in ball detection metrics and allow larger models to generalise better. It would also be more representative of deployment conditions to replace or supplement the offline Roboflow augmentation with augmentations more suitable for broadcast footage (removing vertical flips and 90° rotations, adding motion blur and realistic compression artefacts).

5.5 Final Remarks

Football is still the world game, played and watched by all, irrespective of economic background. The statistical revolution that is transforming professional football at the highest levels offers the potential to benefit every level of the game — but only if the tools that underlie it are made available. This thesis has shown that meaningful object detection from broadcast football footage is possible on consumer hardware using the lightweight versions of the YOLOv8 architecture without expensive infrastructure or specialised engineering expertise.

The results show that a practitioner with a mid-range laptop with a discrete GPU or a modern Apple Silicon SOC can run a real-time detection pipeline with YOLOv8s with good player detection and reasonable ball detection at well above the minimum frame rate for smooth video. A CPU-only machine is not ideal for live analysis but can still be a useful tool for offline post-match processing with the nano model. These are affordable options for small clubs, youth academies and individual coaches that lack the equivalent professional tracking infrastructure and wearable sensor systems available at the elite level.

The gap between what is possible at the top of the sport and what is available to the rest of it is not inevitable. Computer vision pipelines built on openly available models and publicly versioned datasets represent a concrete step toward closing it. This thesis is a contribution to that effort.

REFERENCES

- [1] K. M. Ahmad Yousef, A. El-Haj, B. J. Mohd, N. Dawoud, S. Alnaimar, and A. Saleh. Real-time football match analysis: Leveraging yolo for enhanced object detection and possession tracking. *Procedia Computer Science*, 272:38–45, 2025.
- [2] D. K. Alqahtani, M. A. Cheema, and A. N. Toosi. Benchmarking deep learning models for object detection on edge computing devices. In *Proceedings of the 2024 International Conference on Service-Oriented Computing*, pages 142–150, 2024.
- [3] A. Bochkovskiy, C. Y. Wang, and H. Y. M. Liao. YOLOv4: Optimal speed and accuracy of object detection, 2020.
- [4] N. Dalal and B. Triggs. Histograms of oriented gradients for human detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, volume 1, pages 886–893, 2005.
- [5] Delfos. Bundesliga data shootout dataset. Roboflow Universe, 2022. Accessed: 31/06/2026.
- [6] R. Girshick, J. Donahue, T. Darrell, and J. Malik. Rich feature hierarchies for accurate object detection and semantic segmentation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 580–587, 2014.
- [7] M. R. Hartono, C. A. Sari, and R. R. Ali. Football player tracking, team assignment, and speed estimation using yolov5 and optical flow. *Jurnal Teknik Informatika (Jutif)*, 6:51–62, 2025.
- [8] T. Kershaw. The secretive analytics company helping clubs unearth the next moisés caicedo. *The Times*, 2025. Accessed: December 8, 2025.
- [9] A. Krizhevsky, I. Sutskever, and G. E. Hinton. Imagenet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 25, pages 1097–1105, 2012.
- [10] P. S. S. Markappa, C. O’Leary, and C. Lynch. A review of yolo models for soccer-based object detection. In *Proceedings of the Sixth International Conference on Intelligent Computing in Data Sciences (ICDS)*, pages 1–7, Marrakech, Morocco, 2024.

- [11] Daniel Memmert and Dominik Raabe. *Data Analytics in Football: Positional Data Collection, Modelling and Analysis*. Routledge, 2018.
- [12] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 779–788, 2016.
- [13] J. Redmon and A. Farhadi. Yolo9000: Better, faster, stronger. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7263–7271, 2017.
- [14] J. Redmon and A. Farhadi. YOLOv3: An incremental improvement, 2018.
- [15] M. M. H. Shuvo, S. K. Islam, J. Cheng, and B. I. Morshed. Efficient acceleration of deep learning inference on resource-constrained edge devices: A review. *Proceedings of the IEEE*, 111:42–91, 2022.
- [16] A. Tanapatpiboon, W. Kusakunniran, and W. Limroongreungrat. Deep learning-based detection of players and teams in soccer videos with positioning heatmap generation. *Applied Computing and Informatics*, 2025.
- [17] D. Tshiani. A computer vision framework for multi-class detection and tracking in soccer broadcast footage, 2025. Undergraduate Thesis.
- [18] Ultralytics. YOLOv8 documentation. Ultralytics, 2023. Accessed: 31/06/2026.